

Data Science

Lecture 04: Machine Learning

Prof. Dr. Michael Granitzer

Converted from LNKB-tex part-en-ds-chp4-machine-learning.tex. Builds on Lecture 02 (probability, expectation, sampling) and Lecture 03 (conditional independence, which naive Bayes pushes to its extreme).

Module 1

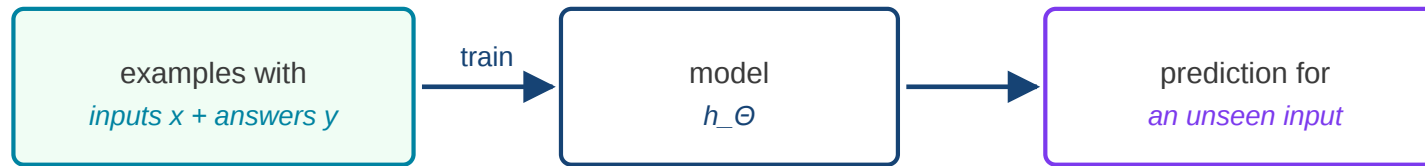
What Is a Learning Problem?

Guiding Question: *What has to be true of a task — and of the data — before "learn it from examples" is a sensible answer?*

Learning From Answers

A radiologist has labelled ten thousand scans. A bank has ten years of loans and knows which ones defaulted. In both cases **every example carries its answer**.

Supervised: every training example carries the answer



The learner is told the correct output for each case it sees.

What information does the learner receive during training?

Speaker notes

Ask before the click: what exactly does the learner see? The answer — an input *and* the output we wanted — is what makes this the easiest setting, and the most expensive one, because somebody had to produce every label.

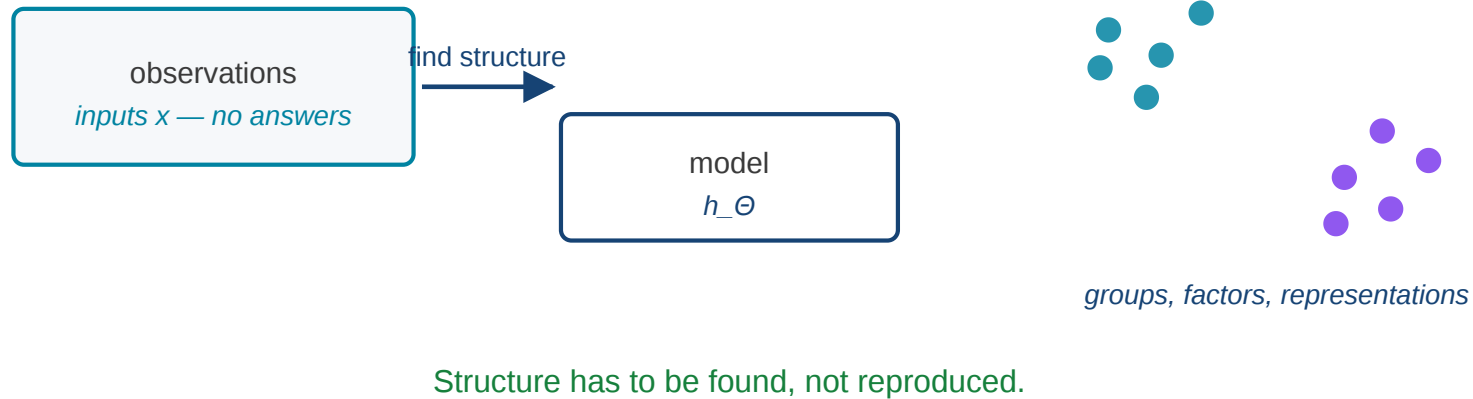
Good examples to name: image classification, medical diagnosis, house prices. The last one previews regression: the "answer" need not be a class.



Learning Without Answers

Now the same bank wants to know **what kinds of customer it has**. Nobody has ever written down the list — the categories are what we are looking for.

Unsupervised: no answers — only the observations themselves



What can we learn when nobody tells us the correct answer?

Speaker notes

Let the room answer first: groups of similar customers, the few dimensions that carry most of the variation, what is unusual. Name clustering, dimensionality reduction, latent structure — but only after they have said it.

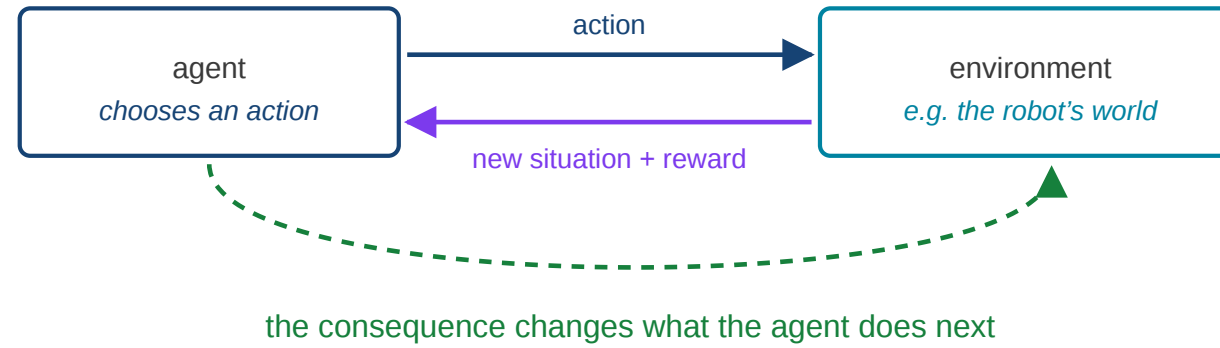
The honest difficulty to flag: without an answer key there is no unambiguous notion of "correct", so evaluation becomes a genuine problem. That returns in the clustering lecture.



Learning by Acting

A robot has to cross a room. Nobody can label the correct motor command for every situation it might find itself in — and it only discovers whether a move was good **after making it**.

Reinforcement: no answer is given — only the consequence of acting



The learner must act to find out whether the action was good.

How can a learner improve when it must act in order to find out whether the action was any good?

Step 1 draws action and consequence; step 2 closes the loop back into the agent's behaviour. Only now name the vocabulary: **agent**, **environment**, **action**, **reward**.

The point that distinguishes this from supervised learning: the feedback says how good the outcome was, not what the right action would have been — and it may arrive long after the action that caused it.

Same Goal, Different Information

Same goal, different information

	WHAT IT IS GIVEN	WHAT FEEDBACK IT GETS
Supervised	inputs + answers	the answer, every time
Unsupervised	inputs only	nothing
Reinforcement	the situation it acts in	a reward, after acting



Reveal one row per click, so the comparison is assembled from the three examples the students have just seen rather than presented as a taxonomy.

Ask which paradigm a spam filter uses (supervised), a recommender trained on clicks (arguably all three), a game engine (reinforcement). The boundaries are softer than the table suggests, and saying so is more honest than a clean three-way split.

Same Goal, Different Information

Same goal, different information

	WHAT IT IS GIVEN	WHAT FEEDBACK IT GETS
Supervised	inputs + answers	the answer, every time
Unsupervised	inputs only	nothing
Reinforcement	the situation it acts in	a reward, after acting

The paradigms differ in **what the learner is given** and **what feedback it gets** — not in what we ultimately want from it.

What Is Common to All Three?

Nobody wrote the rule

The mapping from situation to answer is **too complex or too implicit** to specify by hand.

Examples exist

There is data — or experience — in which the desired behaviour is demonstrated, even if it was never described.

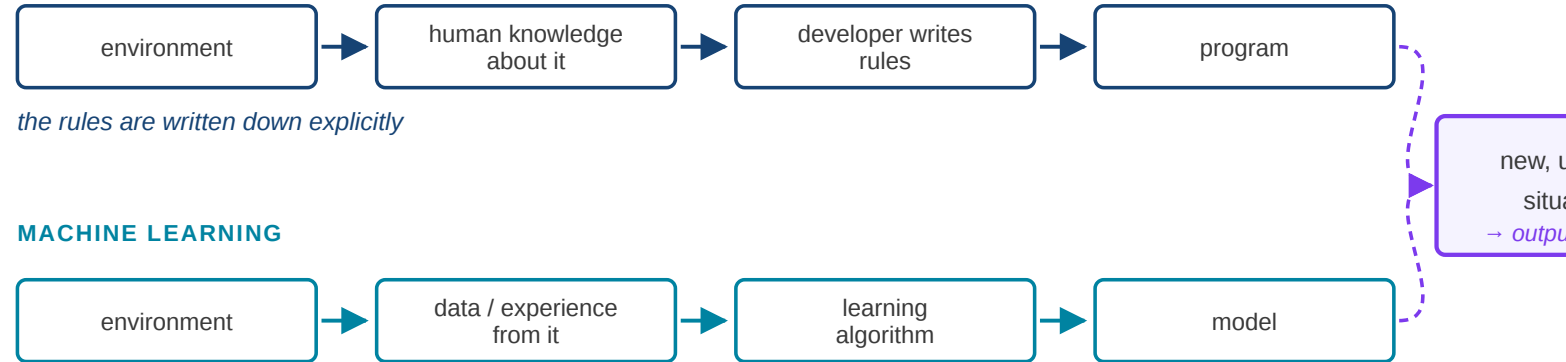
So what replaces the rule that nobody wrote?

This is the bridge to the next slide. Hold the question: in all three paradigms, something is derived from data that a developer would otherwise have had to write down. That "something" is the model — and comparing the two ways of getting it is the next slide.

Two Ways to Get Behaviour

Both are judged on situations neither of them has seen

TRADITIONAL PROGRAMMING



the rules are written down explicitly

MACHINE LEARNING



the rules are inferred from what the environment produced

Speaker notes

Three clicks. First the programming pipeline: the developer holds knowledge about the environment and writes it down as rules. Second the learning pipeline: the environment produces data, and an algorithm derives the model. Third — the point of the slide — **both arrive at the same destination**: a situation neither the developer nor the learner has seen.

Ask between clicks two and three: what is the same in both rows? The environment on the left, and the demand on the right.



What Actually Changed

TRADITIONAL PROGRAMMING

The developer's **knowledge becomes rules**. Deduction: rules plus input give the output.

The assumptions are visible — they are in the code.

MACHINE LEARNING

The environment's **data becomes a model**. Induction: examples plus a learning procedure give a rule.

The assumptions are still there — but now they are in the data, the model family and the objective.

The rules do not disappear when nobody writes them. They become **harder to inspect** — which is why the rest of this lecture is about making every part of that pipeline explicit.

Speaker notes

Deliberate callback to the three modes of reasoning: programming is deductive, learning is inductive. Students who saw that module will recognise the shape, and the guarantee that came with each.

The warning is the honest framing of the trade and sets up the design questions at the end of the module.



What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

Questions first, names second (P4). Let the room produce the three questions in their own words — they always do — and only then attach Mitchell's (1997) vocabulary. A definition that arrives as a label for something already understood costs nothing to remember.

What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

Task T

what the system should learn
to do

Experience E

what it learns from

Performance P

how we decide it improved

Chess



A program that plays, loses, and plays again — millions of times against itself.

- **Task** — play chess well
- **Experience** — games played against itself, and positions from expert play
- **Performance** — fraction of games won; rating against other engines

Speaker notes

The classic example, and worth one observation: the experience here is *generated by the system itself*. No one labelled the good moves — the outcome of the game did.



A Large Language Model



A model shown enormous quantities of text, asked only to continue it.

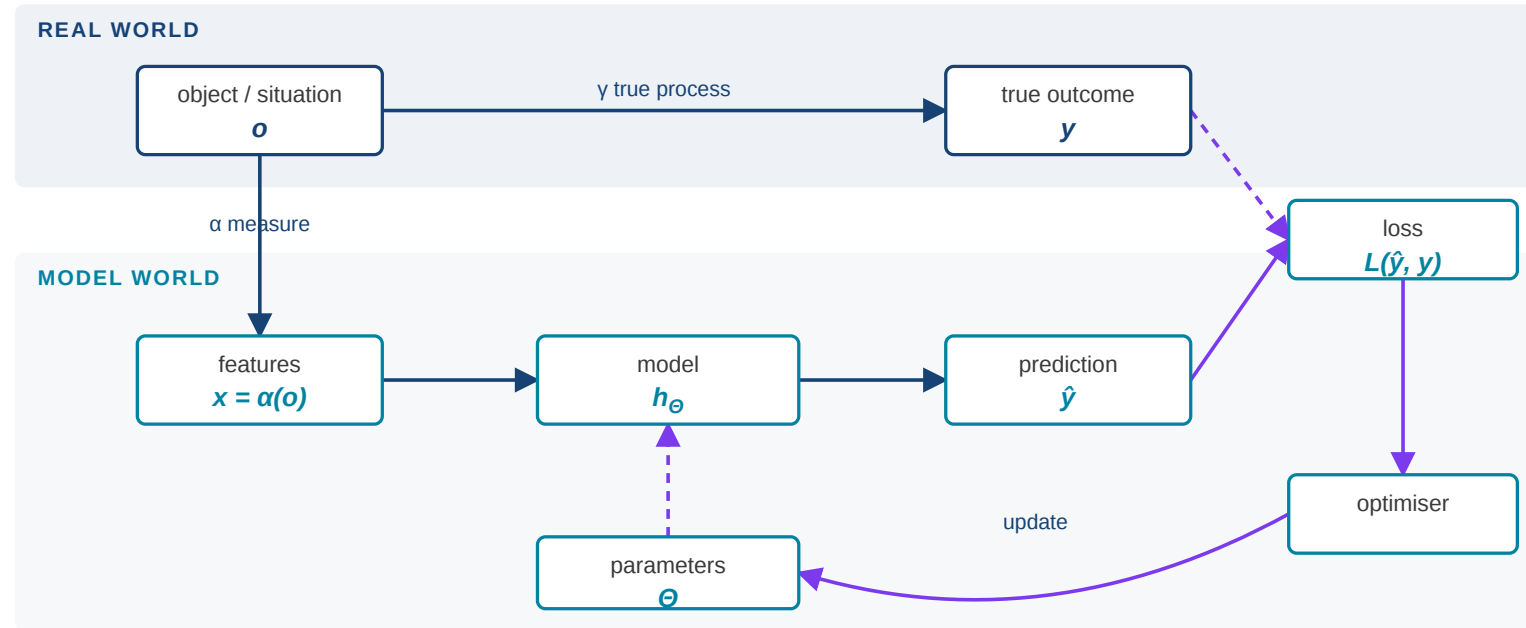
- **Task** — predict the next word
- **Experience** — a very large corpus of text, then human feedback
- **Performance** — likelihood of held-out text; later, human judgement

Two systems with nothing in common — a board game and a text corpus — fit the **same three-part description**. That is what makes it a definition rather than a description of one program.

Pause on the task: "predict the next word" looks far too weak to produce what these systems do. That is exactly why the choice of a *proxy* task is a real design decision — it must force the model to represent what actually matters.

What Would We Ideally Like to Know?

The optimiser changes the parameters — and the training loop closes



There is some real process that turns a situation into an outcome. Why can we not simply write it down?

Speaker notes

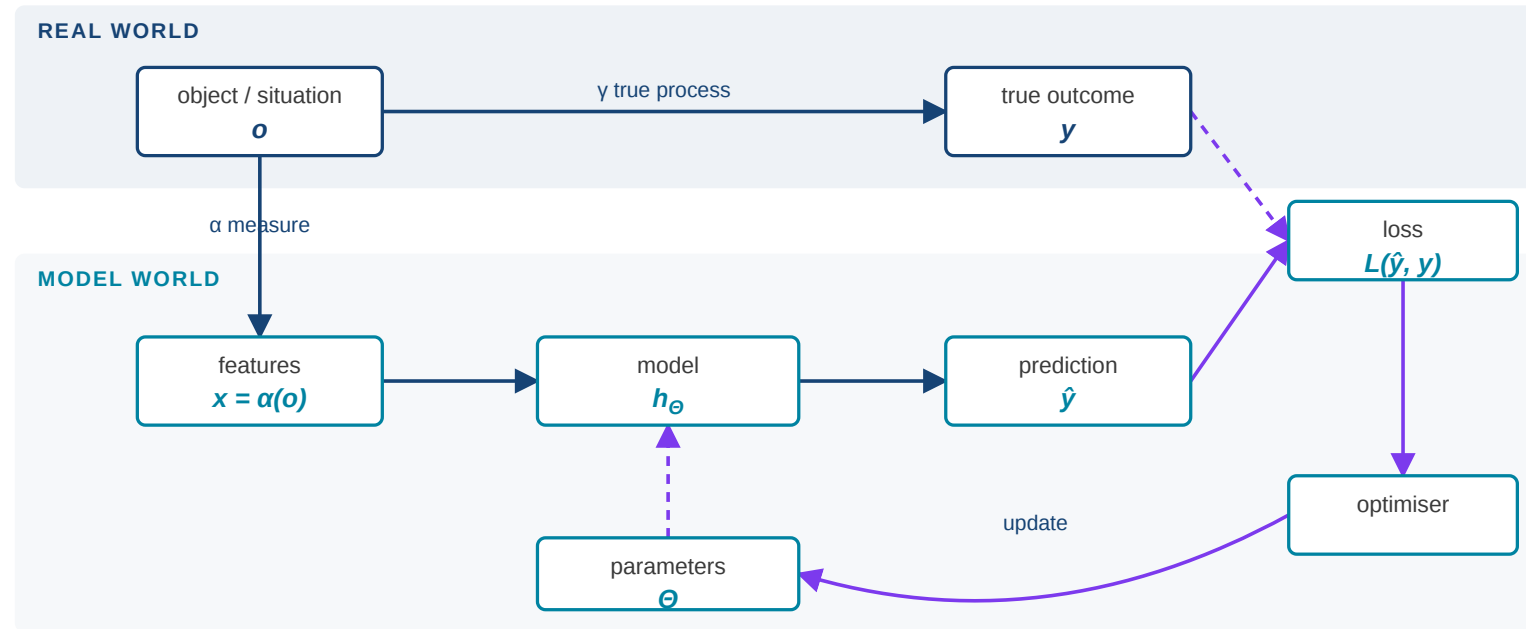
Only the real-world band is drawn: an object or situation o , the true process γ , the outcome y . Let the room answer the question — the process is unknown, or too complex, or not accessible to us at all. That inaccessibility is the reason everything that follows exists.

Notation, kept from the source: γ **is the ideal target function** on real-world objects.



From the Real World to a Model of It

The optimiser changes the parameters — and the training loop closes



Two Worlds, One Correspondence

$$\gamma(o); \rightsquigarrow; h_{\Theta}(\alpha(o)); \equiv; h_{\Theta}(x)$$

γ

the **real process** — unknown,
and not ours to write down

α

the **representation** — what
we chose to measure

h_{Θ}

the **model** — our
approximation, with
parameters Θ

We never learn the world. We build a **model world** and approximate the process inside it.

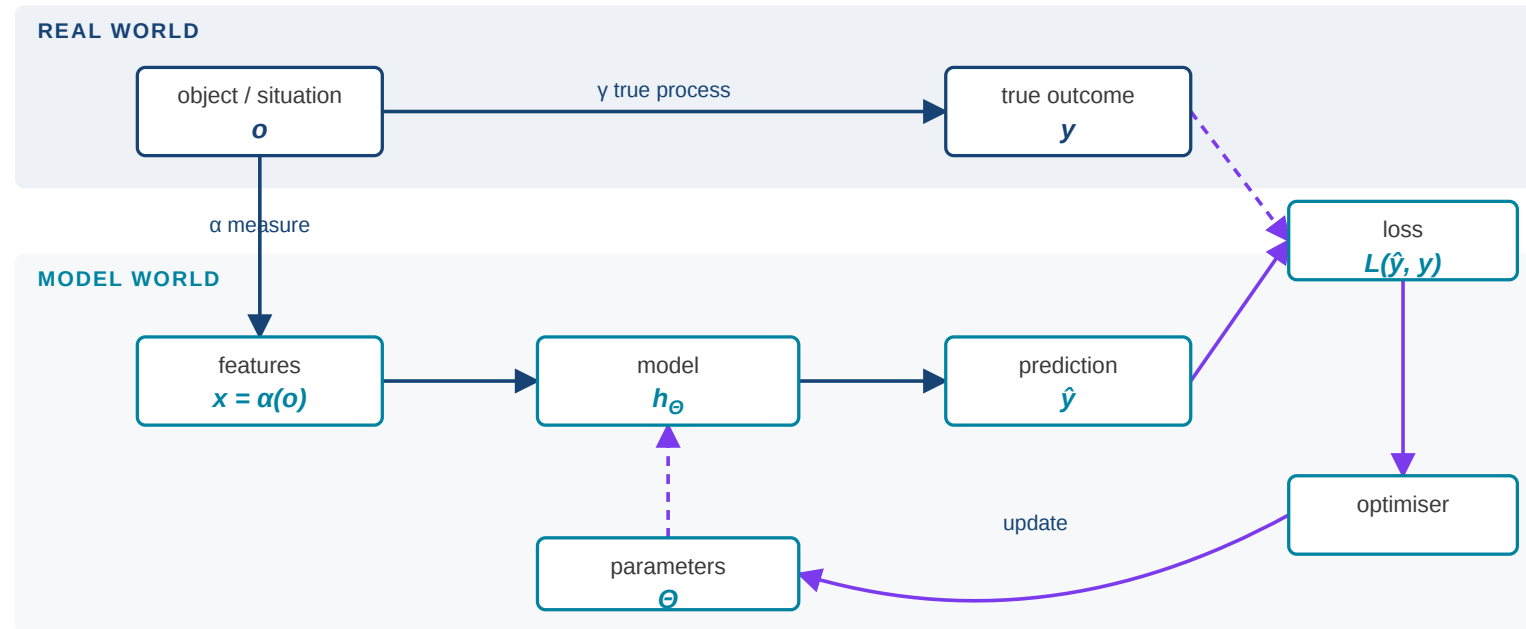
Two clicks: first the measurement — the **model-formation function** α turns an object into features $x = \alpha(o)$ — then the model h_{Θ} and its prediction.

Be precise about the notation, because it is easy to garble: γ is the real process, α **is the representation**, h_{Θ} is the model and Θ are its parameters. α is *not* the parameters.

The relation on the fragment is the heart of the module: two worlds, and a correspondence between them that is only ever approximate. Most practical failures are failures of α — the features did not carry what γ depends on.

Closing the Loop

The optimiser changes the parameters — and the training loop closes



Speaker notes

Two more clicks complete the figure. Click 3 adds the **loss**, which is the first thing on the slide that compares the model world with the real one: it needs $\hat{y}$ *and* the outcome y we actually wanted. Click 4 adds the **optimiser**, which changes the **parameters** Θ — and the loop closes.

Read it aloud once as a sentence: measure the situation, predict, compare with what happened, adjust, repeat. Every method in this course fills these boxes differently.



The Four Choices

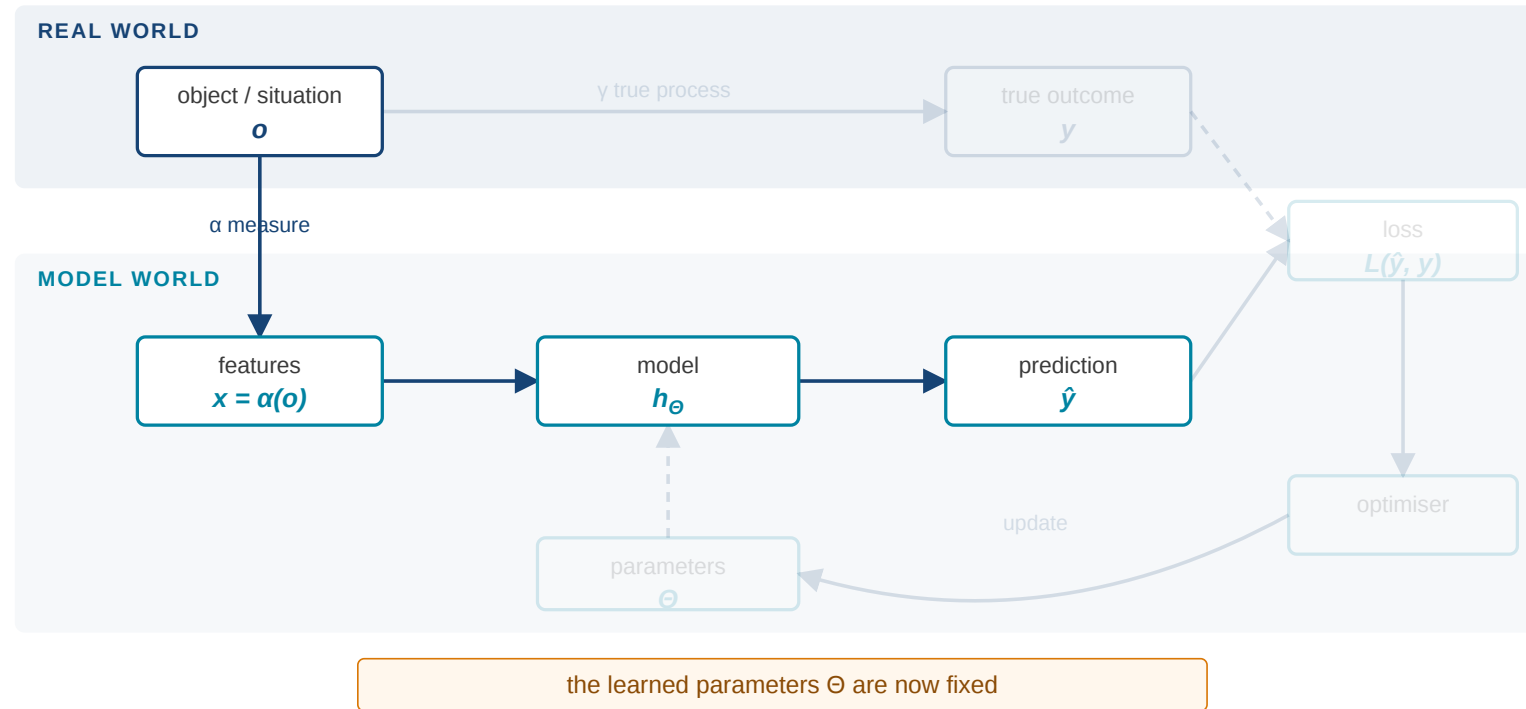


The optimiser has settings of its own that are not learned from the data — **hyperparameters**. Choosing them is part of the outer process, not of the loop.

The compact restatement of the figure, and the map for the whole course: every algorithm later is a different set of these four. Have students place linear regression into the four boxes before moving on.

Training Is Not Using

Inference: a new, unseen situation follows one path — the training machinery is gone



Which of these boxes still exist when the system is actually deployed?

Speaker notes

One click dims everything the training loop needed. What remains is a single path: a new situation, measured, through the model, to a prediction — with Θ now fixed.

Ask the question before clicking. Students routinely assume the loss is somehow still "checking" the model in production; it is not, because in production nobody tells us y . That gap — no answers after deployment — is what monitoring and the evaluation methodology have to work around.



The Task: Predicting Acceleration

The world

O — vehicles

$Y = \mathbb{R}$ — time to accelerate to 100 km/h

$\gamma(o)$ — the true acceleration time of vehicle o

The model world

α — measure wheels, horsepower, weight, torque

X — those four numbers, for every vehicle

h_{Θ} — a linear function of them

Deliberately mundane. The value is that every abstract symbol from the previous unit now has a concrete referent students can argue with — especially α , where "which four numbers" is a real modelling decision.

The Data

wheels	horsepower	weight	torque	acceleration time
4	150	1350	250	9.8
4	320	1600	430	4.9
6	95	2100	210	16.2
...	...	...	...	...

The last column is y the rest is $\mathbf{x}$.

Ask which column is likely useless (wheels, in a passenger-car dataset) and what would happen if it were included anyway — an invitation to regularisation later.



The Remaining Two Pieces

Loss function

Mean squared error — the average of the squared differences between predicted and observed acceleration times.

Learning algorithm

LMS — least mean squares, which adjusts the parameters to reduce that error.

Four choices — representation, model family, loss, optimiser — and the system is fully specified. Every method in this chapter is a different set of four.

A learning problem becomes concrete the moment you can name its **task, experience and performance measure**, and its system becomes buildable the moment you can name its **representation, model, loss and optimiser**.

Close by returning to the four-box map. This unit is the template students should apply when reading any paper: identify the four boxes before judging the result.



Designing the Learning Problem

Before any model is chosen, four things have to be decided — and each of them is a claim about the world.

THE TASK

- ? What should the system predict or decide?
- ? What is one unit of prediction?
- ? What counts as success?

THE DATA

- ? Which observations represent the problem?
- ? How were the examples sampled?
- ? Will future cases look like these?

THE REPRESENTATION A

THE TARGET



? What are the inputs, concretely?

? Which features are available — and which are merely convenient?

? What is lost in the measurement?

? What is γ , in words a domain expert would accept?

? Is the label we have the thing we care about, or a proxy for it?

Every box is a modelling decision that the mathematics will later take for granted. The proxy question in the last box is the one with the most consequences in practice: "did the patient have a disease" versus "was the patient diagnosed", "is this content good" versus "was it clicked".

Designing the System

MODEL AND OBJECTIVE

- ? What does the model family assume, and how flexible is it?
- ? Does minimising this loss correspond to what we actually want?

LEARNING

- ? Which procedure searches the parameters?
- ? How sensitive is the result to initialisation and randomness?

Designing the System — Evaluation

EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

Close the module by pointing at the questions rather than at topics — the roadmap on the next slide then reads as answers to them. The last box is deliberately the one that returns to Lecture 01's opening: a prediction exists to support a decision.



Designing the System — Evaluation

EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

These are the questions the rest of the course answers. Each lecture takes one or two of them and makes them precise.

Module 2

How Learning Is Possible

Guiding Question: *Infinitely many rules fit the data we have seen. What licenses a machine to prefer one of them?*

Learning as Search

The **hypothesis space** $\mathcal{H}$ is the set of all functions the model family can express — every possible value of the parameters Θ . Learning is the search for the best $h_{\theta} \in \mathcal{H}$.

Exhaustive search

Examine all of $\mathcal{H}$. Feasible only when the space is small (with a few special exceptions).

Heuristic search

Use assumptions to explore only part of the space. Efficient, but the optimum may be missed.

Except for very simple problems you cannot have both: searching **all** hypotheses **efficiently and robustly**.

Framing learning as search makes the later algorithms comparable: gradient descent, tree induction and k-means are all heuristic searches through differently shaped spaces.

Inductive Reasoning and Inductive Bias

Inductive reasoning: generalising from seen examples to unseen ones — from observations to a rule that predicts future observations.

Inductive bias: the assumptions by which an algorithm generalises. The choice of hypothesis space is already one; so are assumptions about noise, outliers, the data distribution and the shape of errors.

Without an inductive bias, a system **cannot generalise** at all.

The claim in the box is stronger than students expect and worth defending: with no assumptions, any behaviour on unseen data is consistent with the training data, so nothing can be inferred. Bias is not a defect to be removed, it is the precondition for learning — the question is only whether it is the right bias.

The Bias–Variance Trade-Off

Learning trades **fitting the observed data** against **holding to prior assumptions about unseen data**.

Bias

The tendency to favour particular outcomes regardless of the data — the *stubborn learner*.

High bias → robust to noise, but misses real structure → **underfitting**.

Variance

The tendency to follow irregularities in the data.

High variance → captures patterns, but also noise → **overfitting**.

More parameters generally means more variance. The trade-off is not a defect of particular algorithms — it is a property of inductive inference.

Do not give the decomposition formula here the intuition carries the chapter. Ask which of the two a very deep decision tree suffers from, and which a constant predictor has — the extremes make the axis concrete before any model does.

Takeaway

Learning is a **search** through a hypothesis space, steered by an **inductive bias** you chose — knowingly or not. Every model sits somewhere on the bias–variance axis, and no model escapes it.

Fitting h by Repeated Correction

LMS (least mean squares)

Initialise the weights w .

repeat pick a training example $(\mathbf{x}, y)$ compute the prediction $h_w(\mathbf{x})$ compute the error $e = y - h_w(\mathbf{x})$ update every weight: $w_i \leftarrow w_i + \eta e x_i$ **until** convergence on D

Read the update aloud in words: move each weight in the direction that would have reduced this example's error, in proportion to how much that feature contributed. That sentence is all of gradient descent, before any calculus.

What the Update Rule Encodes

Direction

The sign of e decides whether the prediction goes up or down.

Attribution

The factor x_i gives each feature credit proportional to its value.

Step size

The learning rate η trades speed against stability.

Too large an η oscillates or diverges too small an η converges slowly. It is a hyperparameter — chosen, not learned.

Ask what happens with η far too large before showing it the prediction that "it overshoots and gets worse" is usually made correctly, which earns the hyperparameter concept. The visualisation in the optimisation unit shows it.

Module 3

Linear Regression

Guiding Question: Why squared error — and what exactly does "linear" forbid?

Classification versus Regression

Classification

Y is a finite set of labels.

Will this customer default — yes or no?

Regression

$Y = \mathbb{R}$, the target is a quantity.

What credit line should this customer receive?

The learning machinery is the same only the shape of Y — and therefore the loss — changes.

Keep the same running example across both to make the point that the task, not the data, decides which you are doing. The credit example returns in logistic regression, where a regression model is pressed into classification.

The Model

Linear regression predicts the target as a weighted sum of the features:

$$h_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1x_1 + \dots + w_px_p = \mathbf{w}^\top \mathbf{x}$$

with $x_0 = 1$ absorbing the intercept.

The hypothesis space is $\mathbb{R}^{p+1}$: one hypothesis per weight vector.

Point back at the hypothesis-space unit — here it is a continuous space of finite dimension, which is why calculus rather than enumeration finds the optimum.

The Objective: Residual Sum of Squares

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w} \in \mathbb{R}^{p+1}} \text{RSS}(\mathbf{w}), \quad \text{RSS}(\mathbf{w}) = \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

RSS measures **effectiveness** — how good the predictions are. Do not confuse it with **efficiency**, which is about runtime and computational effort.

The effectiveness/efficiency distinction is from the source and worth keeping; students routinely say "efficient model" when they mean "accurate model".

One Feature: The Closed Form

For a single feature the minimiser can be written down directly:

Slope

$$\hat{w}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

The covariance of x and y over the variance of x .

Intercept

$$\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x}$$

The fitted line passes through the point of means.

Both formulas are built from quantities the students defined in the statistics module — covariance and variance. Say that explicitly: regression is not new mathematics, it is the old summary statistics arranged to answer a prediction question.

How Much Variance Is Explained?

$$\text{TSS} = \text{ESS} + \text{RSS}, \quad R^2 = \frac{\text{ESS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}}$$

where TSS is the total, ESS the explained and RSS the residual sum of squares.

R^2 is the fraction of the target's variation the model accounts for: 0 means no better than predicting the mean, 1 means an exact fit.

Training R^2 never decreases when a feature is added, however useless the feature. A high training R^2 is not evidence that a model generalises — test R^2 and adjusted R^2 can and do fall.

Speaker notes

Connect to the baseline unit: $R^2 = 0$ is exactly the trivial-regression baseline. The warning is the regression counterpart of training error — and the motivation for the cross-validation the students already know.



Higher Dimensions

With p features the same objective in matrix form,

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

gives the optimum in one step whenever $\mathbf{X}^\top \mathbf{X}$ is invertible.

Invertibility fails when features are linearly dependent or when $p > n$ — more unknowns than equations. Both are common in practice, and both are why regularisation exists.

Do not derive the normal equations state them and spend the time on when they break. The $p > n$ case is the modern one — genomics, text — and motivates the next chapters more than the algebra does.

Takeaway

Linear regression fits a weighted sum by minimising squared residuals. Its solution is built from **covariance over variance**, and R^2 reports how much variation it explains — not whether it will generalise.

Linear in the Parameters, Not in the Features

A model is **linear** when it is linear in the **weights** $\mathbf{w}$. The features may be arbitrary transformations of the input.

Plain

$$h(\mathbf{x}) = w_0 + w_1x$$

a straight line

Polynomial

$$h(\mathbf{x}) = w_0 + w_1x + w_2x^2$$

a curve — still a *linear model*

Basis functions

$$h(\mathbf{x}) = \sum_j w_j \phi_j(\mathbf{x})$$

any fixed feature map ϕ

Speaker notes

This is the slide that unlocks the rest: the same least-squares machinery fits curves, interactions and periodic patterns, as long as the transformation is chosen in advance. Ask what is *not* expressible — anything where the transformation itself must be learned, which is the neural-network module.



Consequences

The good

The closed-form solution and all statistical guarantees carry over unchanged.

The catch

The feature map is a **modelling decision**. Choose too rich a basis and the model overfits too poor and it underfits.

Increasing polynomial degree is a dial on the bias–variance axis you can watch turning.

Tie back to principles: the basis is the inductive bias made explicit. This is the cleanest example in the course of choosing a hypothesis space deliberately.

The Question Nobody Asks

We minimised the *squared* residuals. Why squares — why not absolute deviations, or fourth powers?

Guiding question. Most students have never been given a reason beyond differentiability. There is a real one, and it is worth the slide.

Geometric View

The predictions $\mathbf{X}\mathbf{w}$ live in the **column space** of $\mathbf{X}$ — everything the features can express. The observed $\mathbf{y}$ generally does not.

Least squares picks the point of that space **closest to $\mathbf{y}$ in Euclidean distance**: the orthogonal projection. The residual is perpendicular to every feature.

Consequence

The residual carries no remaining linear information about the features.

Speaker notes

The orthogonality condition is the geometric content of the normal equations. It also explains why adding a feature can never increase RSS: the space you project onto only grew.



The Gauss–Markov Theorem

Gauss–Markov (informal). Among all estimators that are

- **linear** in the observed outputs $(y_1, \dots, y_n)$, and
- **unbiased**,

the least-squares estimator has the **lowest variance**. It is the *Best Linear Unbiased Estimator* — BLUE.

Best

lowest variance among the candidates

Linear

a linear function of the observations

Unbiased

right on average over repeated samples

Speaker notes

Spell out what the theorem does *not* say: nothing about non-linear or biased estimators — and biased estimators (ridge, lasso) can beat OLS in prediction error. That is not a contradiction, it is a different criterion.



What This Buys, and What It Costs

The payoff

Minimising RSS yields optimal statistical properties under **very mild assumptions** — no normality needed, only uncorrelated errors of constant variance.

The price

Alternatives — absolute error, Huber loss — are **more robust to outliers** but give up the BLUE guarantee.

Squared error is not an arbitrary convenience: it is the loss whose minimiser is the best linear unbiased estimator. Departing from it is legitimate — but it is a trade, and you should be able to name what you traded.

Close the loop with the robustness discussion from descriptive statistics: mean versus median reappears here as squared versus absolute loss. Same trade-off, new context — exactly the spiral the pedagogic concept asks for.

Two Ways to the Same Minimum

Direct (closed form)

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Exact, one step. Requires forming and inverting a $(p+1) \times (p+1)$ matrix — costly for large p , impossible when it is singular.

Iterative (gradient descent)

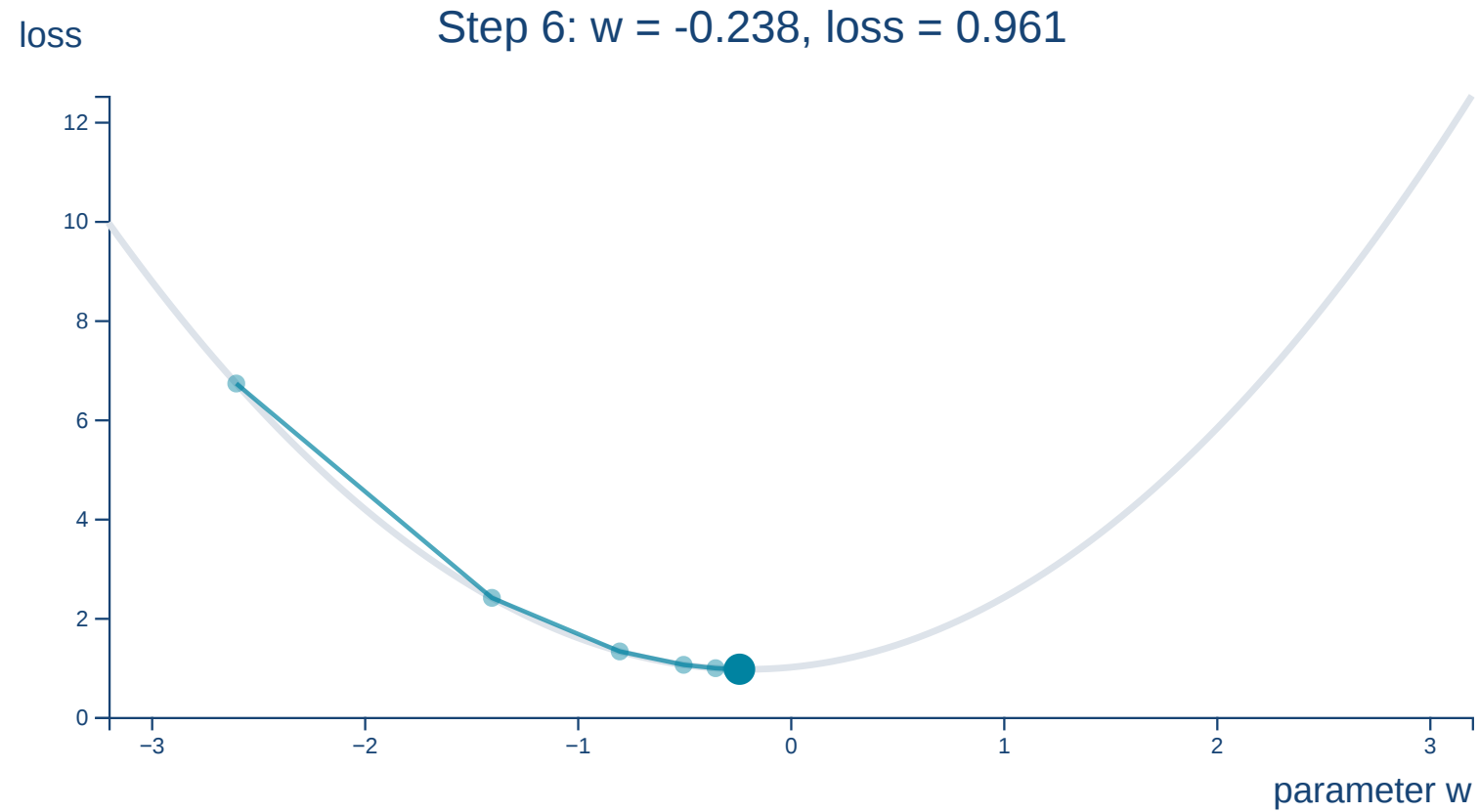
$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \text{RSS}(\mathbf{w})$$

Approximate, many steps. Scales to large p and n , works when the direct solution does not exist, and extends to models with no closed form at all.

The iterative rule is exactly the **LMS algorithm** from Module 2: the update $w_i \leftarrow w_i + \eta e x_i$ is a step along $-\nabla \text{RSS}$, written per example instead of per dataset.

The right-hand column is the one that generalises: logistic regression and neural networks have no closed form, so the iterative method is the only one that survives the rest of the course. Make the LMS identification explicit — students otherwise carry two unconnected procedures.

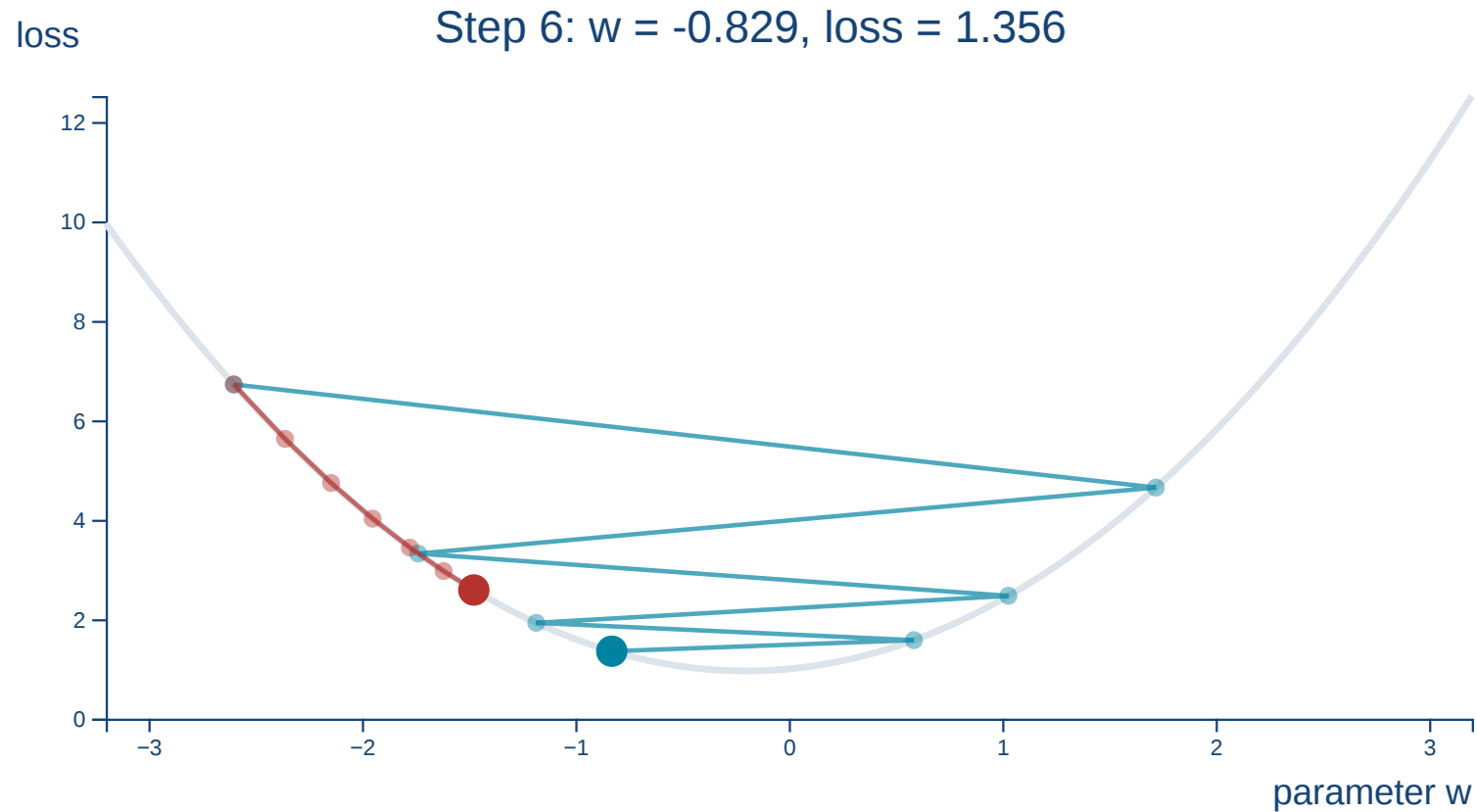
Watching Gradient Descent Work



Stepped viz (P6). Each click is one update. Ask before the first click where the steps will be largest — where the slope is steepest, far from the minimum — and why they shrink as the minimum approaches, without η changing.



The Learning Rate Decides Everything



Teal: $\eta = 0.9$ — overshoots and oscillates across the minimum. Red: $\eta = 0.05$ — stable, but barely moves.



The learning rate is a **hyperparameter**: it is chosen, not learned, and both failure modes are silent — one looks unstable, the other looks converged.

Speaker notes

Predict-then-check: ask which curve belongs to which η before revealing. The "looks converged" failure is the dangerous one — a slow run and a finished run produce the same-looking loss curve over a short window.



Takeaway

Use the **closed form** when the matrix is small and well-conditioned use **gradient descent** when it is not — which, for everything after this unit, is always.

Module 4

From Regression to Classification

***Guiding Question:** A decision is a yes or a no. What goes wrong when we fit a straight line to one?*

Fitting a Line to Labels

Encode the two classes as $y \in \{-1, +1\}$, fit a linear model, and classify by the sign of the prediction:

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

One feature

The **discrimination point** x' satisfies

$$w_0 + w_1 x' = 0$$

Everything to one side is one class.

Two features

The **discrimination line** satisfies

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

In p dimensions it becomes a hyperplane.

The decision boundary is where the model output crosses zero — a level set. Say it once here it holds for logistic regression, SVMs and a neural network's final layer alike.

Where It Breaks

Squared loss punishes correctness

A point far on the *correct* side has a large residual, so least squares drags the boundary toward it.

The boundary shifts

Adding well-classified points — or an outlier — moves the discrimination point, even when nothing was misclassified.

The output is also unbounded: $\mathbf{w}^\top \mathbf{x}$ can be 17 or -40 , and neither is interpretable as a degree of belief.

Speaker notes

Force the interpretation (P2): ask why "being very right" should ever count as error. Once students see that squared loss is the wrong loss for a decision, the sigmoid arrives as a fix rather than a formula.



Two Problems, One Fix

$\text{sign}(\mathbf{w}^\top \mathbf{x})$ is not differentiable

No gradient means no gradient descent — the optimisation machinery does not apply.

The boundary is unstable

Squared loss on ± 1 labels lets well-classified points shift the discrimination point.

Replace the hard threshold with the **logistic sigmoid**:

$$h_{\Theta}(\mathbf{x}) = \text{expit}(\mathbf{w}^\top \mathbf{x}) \approx P(y = 1 \mid \mathbf{x})$$

The output becomes a smooth, bounded quantity that can be read as a probability — and can be differentiated.

Both problems have the same root: a decision is a step function, and steps have no useful derivative. Softening the step is the standard move; it returns as the activation function in neural networks.

Odds and Log-Odds

Odds

The ratio of favourable to remaining cases.

Rolling a six: odds 1 : 5.

Logit

The **log-odds** of a probability p :

$$\text{logit}(p) = \ln \frac{p}{1 - p}$$

Range

$p \in (0, 1)$ maps to $(-\infty, +\infty)$
— exactly the range a linear model produces.

The modelling assumption: the logit of the target is a linear combination of the features.

$$\ln \frac{P(y = 1 \mid \mathbf{x})}{1 - P(y = 1 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x}$$

Speaker notes

This is the cleanest way in: we do not force a probability to be linear (it cannot be — it is bounded), we make its log-odds linear. The sigmoid is then not an arbitrary squashing function but the inverse of the logit.



The Model

Inverting the logit gives the **logistic sigmoid**:

$$h_{\Theta}(\mathbf{x}) = \text{expit}(\mathbf{w}^{\top} \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^{\top} \mathbf{x}}}$$

Shape

An S-curve: near-linear around 0, saturating towards 0 and 1 at the extremes.

Reading

$h_{\Theta}(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$, and the decision boundary is still $\mathbf{w}^{\top} \mathbf{x} = 0$ — where the sigmoid passes 0.5.

Speaker notes

The boundary is unchanged from linear regression for classification what changed is the *loss* and the interpretability of the output. Worth stating explicitly, since students often expect a curved boundary.



The Loss: Cross-Entropy

Binary cross-entropy (log loss)
$$L(\theta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln h_{\theta}(\mathbf{x}_i) + (1 - y_i) \ln (1 - h_{\theta}(\mathbf{x}_i)) \right]$$

For $|Y| > 2$ classes, the multiclass form sums over the classes with a softmax output.

Where does it come from? From the **likelihood**: $P(y_i = 1 \mid \mathbf{x}_i; \theta) = h_{\theta}(\mathbf{x}_i)$ and $P(y_i = 0 \mid \mathbf{x}_i; \theta) = 1 - h_{\theta}(\mathbf{x}_i)$. Maximising the likelihood of the data is minimising this loss.

Speaker notes

The derivation is one line and worth doing aloud: write the likelihood of one example as $h^y (1-h)^{(1-y)}$, take the product over examples, take the log, negate. Maximum likelihood is the origin of the loss — not a separate idea.



Why It Fixes the Problem

Confident and correct

Loss goes to zero — no penalty for being emphatically right.

Confident and wrong

Loss goes to infinity — exactly the errors we want punished.

There is no closed-form solution the parameters are found by gradient descent on this loss.

Contrast directly with the squared-loss failure two units back: cross-entropy punishes confident mistakes and ignores confident successes, which is what a classification loss should do.

Takeaway

Logistic regression is a **linear model of the log-odds**, trained by maximising likelihood. It keeps the linear decision boundary but produces a calibrated probability — and a loss that punishes the right kind of error.

Module 5

Probabilistic Classification

Guiding Question: *The data tells us how attributes behave given the class. We need the reverse. What does the conversion cost?*

Thinking With Uncertainty

Bayes' theorem lets us update a **hypothesis** H in the light of **data** D :

$$P(H \mid D) = \frac{P(H) \cdot P(D \mid H)}{P(D)}$$

This is the whole apparatus of evidence-based decision making in one line: a belief, a measurement of how well the belief explains what we saw, and a normalisation.

Speaker notes

Deliberate second pass over Bayes (P4/P5 decision rule: this is consolidation, so math first). The new element is the reading of H and D as hypothesis and data, which is the form used by every learning algorithm later in the course.



Example: Do I Have the Flu?

You have a headache and a cough. From the literature:

Likelihood

90% of people with the flu
show headache and coughing

$$P(D \mid H) = 0.9$$

Prior

5% of people currently have
the flu

$$P(H) = 0.05$$

Evidence

headache and coughing occur
in 20% of people

$$P(D) = 0.2$$

How likely is it that you have the flu?

Speaker notes

Ask for a guess first — most people say something close to 90%, repeating the base-rate error from the probability chapter. 22.5% is the answer.



Example: Do I Have the Flu?

You have a headache and a cough. From the literature:

Likelihood

90% of people with the flu
show headache and coughing

$$P(D \mid H) = 0.9$$

Prior

5% of people currently have
the flu

$$P(H) = 0.05$$

Evidence

headache and coughing occur
in 20% of people

$$P(D) = 0.2$$

How likely is it that you have the flu?

$$P(H \mid D) = \frac{0.05 \cdot 0.9}{0.2} = 0.225$$

Why So Low?

Few people have the flu

The prior $P(H) = 0.05$ caps how far the evidence can carry us.

The symptoms are common

$P(D) = 0.2$ means many people have these symptoms *without* the flu.

A symptom is informative to the degree that it is **rare in general** but **common under the hypothesis**. A likelihood alone tells you nothing.

Two reasons, both structural. This is the criterion that reappears as feature informativeness in classification: a feature that fires for everything carries no information. Connect it explicitly — it makes information gain feel inevitable later.

Takeaway

Bayesian decision making weighs **prior × likelihood** against how ordinary the evidence is. Deciding under uncertainty means keeping all three numbers in view, not just the one the study reports.

Two Directions of the Same Connection

Observed connection

$$A_1, \dots, A_p \mid B$$

Given the class, how do the attributes behave? — the standard situation, and the one data can answer directly.

Reversed connection

$$B \mid A_1, \dots, A_p$$

Given the attributes, which class? — the diagnosis situation, and the one we actually need.

$$P(B \mid A_1, \dots, A_p) = \frac{P(B) \cdot P(A_1, \dots, A_p \mid B)}{P(A_1, \dots, A_p)}$$

This is the base-rate lesson turned into an algorithm: the data gives us the easy direction, Bayes converts it into the one we need. Ask which direction a labelled dataset supplies — attributes given class, by construction.

What the Classifier Needs

Class priors

$$P(\text{class} = y)$$

How common is each class, before seeing any attribute?

Attribute likelihoods

$$P(\text{attribute} = v \mid \text{class} = y)$$

How does each class distribute its attribute values?

Given both, the posterior $P(\text{class} \mid \text{attributes})$ follows from Bayes — and the prediction is the class with the highest posterior.

Estimating $P(A_1, \dots, A_p \mid B)$ requires the **joint** distribution of all attributes within each class: exponentially many parameters, and never enough data.

Speaker notes

End the unit on the obstacle rather than the solution — the warning is exactly the parameter explosion from the graphical-models lecture, and naive Bayes is the crudest possible independence assumption applied to it.



Generative and Discriminative Classifiers

Generative

Model the **joint** distribution $P(A, B)$ — how the data was produced — then derive $P(B | A)$.

Naive Bayes.

Discriminative

Model $P(B | A)$ **directly** — only the boundary between classes.

Logistic regression.

The distinction runs through the rest of machine learning. A generative model can be sampled from and can handle missing attributes a discriminative model spends all its capacity on the decision. Neither is universally better.

The Naive Assumption

Assume the attributes are **conditionally independent given the class**:

$$P(A_1, \dots, A_p \mid B) = \prod_{j=1}^p P(A_j \mid B)$$

Exponentially many parameters collapse to p per class — one distribution per attribute.

The assumption is almost always false. Naive Bayes nonetheless classifies well, because ranking the classes correctly is easier than estimating their probabilities correctly.

Direct callback to conditional independence: this is the same move as a graphical model, taken to its extreme — a star-shaped network with the class at the centre. The note is the honest part: the probabilities are miscalibrated, the argmax often is not.

The Classifier

$$P(B_i \mid A_1, \dots, A_p) = \frac{P(B_i) \cdot \prod_{j=1}^p P(A_j \mid B_i)}{\sum_{i=1}^k P(B_i) \cdot \prod_{j=1}^p P(A_j \mid B_i)}$$

Score each class

prior × product of attribute likelihoods

Normalise

divide by the total over all classes — the theorem of total probability

Predict

the class with the largest posterior

For prediction alone the denominator can be skipped — it is the same for every class. It is needed only when the probability itself is reported.

The three-step structure is the whole algorithm. Point out that the denominator is total probability, which the students proved in the probability lecture — nothing new is being introduced here, only assembled.

Example: Enjoy Sport?

Outlook	Temperature	Humidity	Wind	EnjoySport
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rain	mild	high	weak	yes
rain	cool	normal	weak	yes
rain	cool	normal	strong	no

Estimate the priors and the per-attribute likelihoods by **counting**, then score a new day.

Speaker notes

Have the group compute $P(\text{yes})$ and $P(\text{sunny} \mid \text{yes})$ aloud — the estimation is pure counting, which is what makes the method so cheap. Then ask what happens for $P(\text{overcast} \mid \text{no})$: zero, which kills the entire product.



The Zero-Frequency Problem

If an attribute value never occurs with a class in the training data, its estimated likelihood is **zero** — and one zero factor drives the whole product to zero, whatever the other attributes say.

Laplace smoothing adds a pseudo-count to every combination, so that unseen does not mean impossible.

One missing count

vetoed every other piece of evidence

Naive Bayes buys tractability with a **conditional independence assumption it knows to be false** — and works anyway, because classification needs the right argmax, not the right probability.

Speaker notes

Close on the epistemic point (P3): a knowingly wrong assumption can still yield a useful model, provided you know which conclusions it does and does not support. Calibrated probabilities are exactly what it does not support.



Module 6

Measuring Performance

Guiding Question: *A model reports 99.9% accuracy and has never detected a single case. What should we have measured instead?*

What We Want to Know

The **true misclassification rate** of a hypothesis h is the probability that it errs on a randomly drawn object:

$$\text{err}^*(h) = P(h(\mathbf{x}) \neq y(\mathbf{x}))$$

It is defined over the **whole distribution of objects** — which we cannot observe.

Speaker notes

Deliberately parallel to the population/sample distinction from the statistics module: err^* is a population quantity. Students who see the analogy will not be surprised by cross-validation.



What We Can Measure

We estimate $\text{err}^*(h)$ by the error on a set of examples $D_s \subseteq D$ whose true labels are known:

$$\text{err}(h, D_s) = \frac{1}{|D_s|} \sum_{\mathbf{x} \in D_s} 1[h(\mathbf{x}) \neq y(\mathbf{x})]$$

The estimate is only as good as the sample. Which examples land in D_s — and whether h has seen them before — decides whether the number means anything.

Everything in the next unit follows from that warning. Ask what happens if D_s is the data the model was fitted on the answer motivates hold-out estimation without needing to be told.



Training Error Is Not an Estimate

When the evaluation set equals the training set, $D_{tr} = D$, the resulting number measures how well the model **memorised**, not how well it will generalise.

Training error is a lower bound on the true error, and an arbitrarily bad one. A model with enough capacity can drive it to zero while learning nothing.

Connect straight to the bias–variance unit: training error falls monotonically with capacity, which is exactly why it cannot be the criterion for choosing capacity.

Four Estimation Schemes

Hold-out (2-fold)

Split D into a training and a test part, $D_{tr} \subset D$. Fit on one, evaluate on the other. *Simple, but wastes data and depends on the split.*

k -fold cross-validation

Partition D into k folds; each fold serves once as the test set. Average the k errors. *Uses all data for both roles $k = 5$ or 10 is customary.*

Leave-one-out (n -fold)

$k = n$: every example is a test set of size one. *Nearly unbiased, expensive, and high variance.*

Bootstrapping

Resample D with replacement to build training sets evaluate on the examples left out. *Useful when D is small.*

Ask which scheme a Kaggle-style leaderboard implements, and what repeatedly consulting a fixed test set does to its status as an estimate — the test set becomes training data for the modeller.

Misclassification Costs

Not every error costs the same. A missed diagnosis and a false alarm have different consequences, so counting errors equally is itself a modelling assumption.

A **cost matrix** assigns a cost to each (true class, predicted class) pair; the objective becomes expected cost rather than error rate.

Speaker notes

This is the hinge into precision and recall in two units' time. Ask the group to set the two costs for a spam filter and then for a cancer screen the ratios they choose differ by orders of magnitude, and both are defensible.



Takeaway

An error number is meaningless without the sampling scheme that produced it. **Never evaluate on data the model was fitted on**, and state the cost model when errors are not interchangeable.

A Number Alone Means Nothing

"95% accuracy" is not a result until you know what a trivial method achieves.

Trivial classification

Always predict the **majority class**.

Trivial regression

Always predict a **central tendency** — the mean or median of y .

Random guessing

Draw a label from the marginal distribution $p(y)$.

Insist on the habit: report the baseline next to the model, always. This is the cheapest defence against self-deception in the whole course.

Example: Why Accuracy Deceives

A screening test for a disease affecting **1 in 1,000** people.

The "model"

Always predict *healthy*.

Its accuracy

99.9% — and it has never detected a single case.

On imbalanced data the majority baseline is nearly unbeatable by accuracy, and completely useless. The metric, not the model, is the problem.

The same base-rate structure as the Bayes unit, now as an evaluation trap. This is the motivation for precision and recall, which follow immediately.

Why the Error Rate Is Not Enough

For multiclass classification $h_{\Theta} : X \rightarrow Y$ with $Y \subseteq \{1, \dots, N\}$, the plain error rate:

Fails on imbalance

The majority baseline scores high while detecting nothing.

Hides the distribution

Which classes the errors fall on stays invisible.

Hides the kind of error

Missing assignments and imprecise assignments count the same.

Three distinct failures, deliberately separated: imbalance, distribution across classes, and type of error. Each one motivates a different part of what follows.



The Confusion Matrix

Rows are true classes, columns are predictions cell (i, j) counts the examples of class i that h assigned to j . The diagonal is correct, and every off-diagonal cell names a specific confusion.

The matrix is the raw material: every measure below is a ratio of its cells. Read it before reading any summary number.

On a well-known dataset the off-diagonal structure is often interpretable — which two classes get confused tells you something about the representation, not just the score.



Four Counts, Per Class

For a class j , every prediction falls into one of four cells:

TP_j — **true positives**

Assigned to j **and** truly in j .

FP_j — **false positives**

Assigned to j but **not** in j .

FN_j — **false negatives**

Not assigned to j but truly in j .

TN_j — **true negatives**

Not assigned to j and not in j .

Read the letters as **decision then correctness**: P/N is what h decided, T/F is whether that decision was right.

The mnemonic in the tip box removes nearly all of the confusion students have with these four terms. Have them classify a concrete example aloud before moving on.

Precision and Recall

Recall — completeness

$$\text{rec}_j = \frac{TP_j}{TP_j + FN_j}$$

Of everything that truly belongs to j , what fraction did h find?

Precision — exactness

$$\text{prec}_j = \frac{TP_j}{TP_j + FP_j}$$

When h says j , how often is it right?

Both are **conditional probabilities with different conditions** — recall conditions on the truth, precision on the prediction. Swapping them is the same error as swapping the conditionals in Bayes' theorem.

Speaker notes

The link to Bayes is the payoff of teaching probability first: precision and recall are $P(\text{correct} \mid \text{predicted})$ and $P(\text{found} \mid \text{true})$. Ask which one a search engine optimises and which one a screening programme optimises.



Mini-Exercise

A spam filter processes 1,000 emails, 100 of which are spam. It flags 80 emails, of which 60 are genuinely spam.

Compute precision and recall. Which one would you rather improve, and why?

Mini-Exercise — Solution

The final sentence is the one to stabilise — the choice between precision and recall is a statement about costs, which is why the cost-matrix slide came earlier.

Mini-Exercise — Solution

$$\text{Precision} = 60/80 = 0.75$$

A quarter of flagged mail is legitimate.

$$\text{Recall} = 60/100 = 0.60$$

Forty spam messages still arrive.

For a spam filter, **precision** usually matters more: a lost legitimate email costs far more than a spam message that slips through. The cost matrix decides, not the metric.

Takeaway

The confusion matrix shows **which** errors occur precision and recall split "being wrong" into **being imprecise** and **being incomplete**. Which one you optimise is a decision about costs, not about mathematics.

Squared Error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (h(\mathbf{x}_i) - y_i)^2 \quad \text{RMSE} = \sqrt{\text{MSE}}$$

MSE

Average quadratic loss always non-negative, lower is better.

Unit is the **square** of the target's unit — hard to interpret directly.

Large errors are penalised disproportionately.

RMSE

Same ordering as MSE, but back in the **target's own unit** — which is why it is the one usually reported.

Speaker notes

The quadratic penalty is a modelling choice with consequences: it makes the fit sensitive to outliers, which connects back to robustness in descriptive statistics. Ask when that sensitivity is desirable — when large errors really are disproportionately costly.



Absolute and Relative Error

MAE

$$\frac{1}{n} \sum_i |h(\mathbf{x}_i) - y_i|$$

Easier to interpret than RMSE
treats all errors proportionally.

Mean error

Drop the absolute value to
keep the **direction** — reveals
systematic over- or under-
prediction.

Mean relative error

$$\frac{1}{n} \sum_i \frac{|h(\mathbf{x}_i) - y_i|}{|y_i|}$$

The average error *per unit of target* — comparable across
scales.

Normalised RMSE variants aim to make errors comparable between datasets, but are hard to interpret. Prefer a measure whose unit you can name.

Speaker notes

The mean error (with sign) is underused and worth highlighting: a model can have excellent RMSE and still be biased in one direction, which matters for forecasting and pricing.



Takeaway

Squared measures punish large errors and lose the unit absolute measures stay interpretable signed measures expose bias. Report the one whose **unit and failure mode** match the decision the prediction feeds.